Reinforcement Learning

Class 2

The full reinforcement learning:

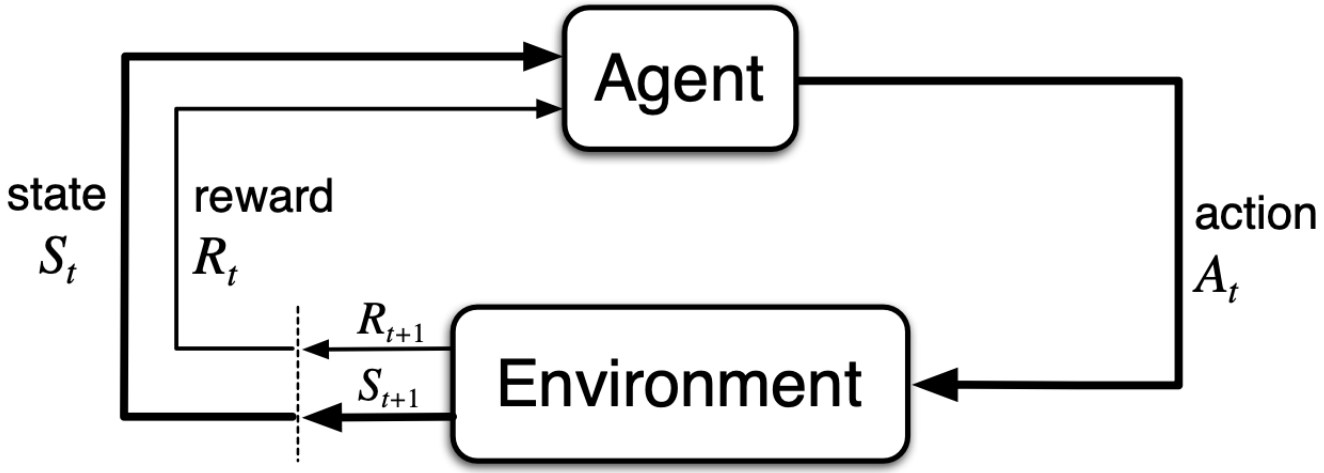
Markov Decision Processes

Mostly based on D. Silver (UCL) lecture on MDPs from 2015
Available in YouTube

Context

- Unlike in the previous session, we will look at scenarios that are **associative** – optimal actions change in different situations (states)
- Still sequential decision making, but actions **influence** not just immediate **rewards**, but also **subsequent states**, and so future rewards
- Instead of estimating $q_*(a)$ we will now estimate $q_*(s,a)$, where s stands for the current state and a for the action taken
- We will look at the **full reinforcement learning problem**, looking a mathematical framework to model these problems

Framework: agent-environment



Agent – represents the learner and decision maker, which interacts making actions (A_t) over the environment

Environment – everything outside the agent - reacts to actions by changing its internal state (communicated to the agent- S_t) and issuing **rewards** (R_t), that the agent will seek to maximize over time

We will assume here time is finite and steps are discrete (t = 0, 1, 2, 3, …, T)

Trajectory/ episode: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, \ldots, A_{T-1}, R_T, S_T$

Markov Decision Processes and RL

- **Markov Decision Processes** (**MDPs**) formally define an environment for RL
- Imply that the environment is **fully observable** – the current state characterizes the process (or we will decide based on what we can observe)
- Most RL problems can be formalized as MDPs
  - K-armed bandits – MDPs with a single state
  - Optimal control (continuous extension)
  - Partially observable (can be extended)
- We will define MDPs and see how they model the full RL problem, starting by defining Markov processes/ chains, adding rewards, and actions to reaching the full RL scenario

Examples - Bioreactor

- **Aim**: set moment-by-moment temperatures and stirring rates for a bioreactor (vat containing nutrients and bacteria used to produce useful chemicals)
- **Actions**: target temperatures and stirring rates (vector)
- **States**: sensory readings (e.g. oxygen/CO2 rates), symbolic inputs representing the ingredients in the vat and the target chemical (vector)
- **Rewards**: measure of the rate at which the useful chemical is produced by the bioreactor (real number)

Examples – Pick-and-place robot

- **Aim**: control the motion of a robot arm in a repetitive pick-and-place task
- **Actions**: voltages applied to each motor at each joint
- **States**: readings of joint angles and velocities.
- **Rewards**: +1 for each object successfully picked up and placed; to encourage smooth movements, on each time step a small, negative reward can be given as a function of the moment-to-moment “jerkiness” of the motion

Markov property

- In a Markov Process, the probability of each possible state at time t (S_{t+1}) depends only on the immediately preceding state (S_t)

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]$$

- This means that the current state must include information about all aspects of the past that make a difference for the future - **Markov property**
- "The future is independent of the past given the present" (D. Silver) – once the state is known all the past may be thrown away

Markov processes

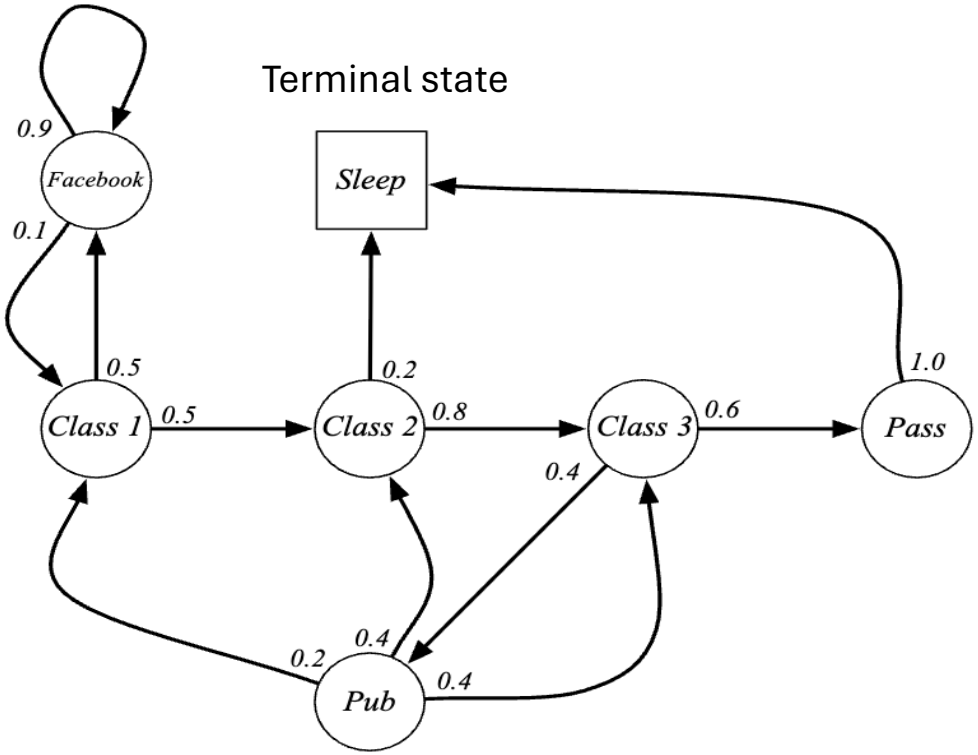
- A **Markov Process** or a **Markov chain** is a memoryless random process – sequence of random states S_1, S_2, … following the Markov property
- A Markov Process is a tuple **(S, P)**
  - S is a finite set of states (S_1, S_2, …)
  - ***P*** is a state transition probability matrix where each element:

$$\mathcal{P}_{ss'} = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s\right]$$

$$\begin{bmatrix} \mathcal{P}_{11} & \cdots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \cdots & \mathcal{P}_{nn} \end{bmatrix}$$

where each row sums to 1

Markov process - example



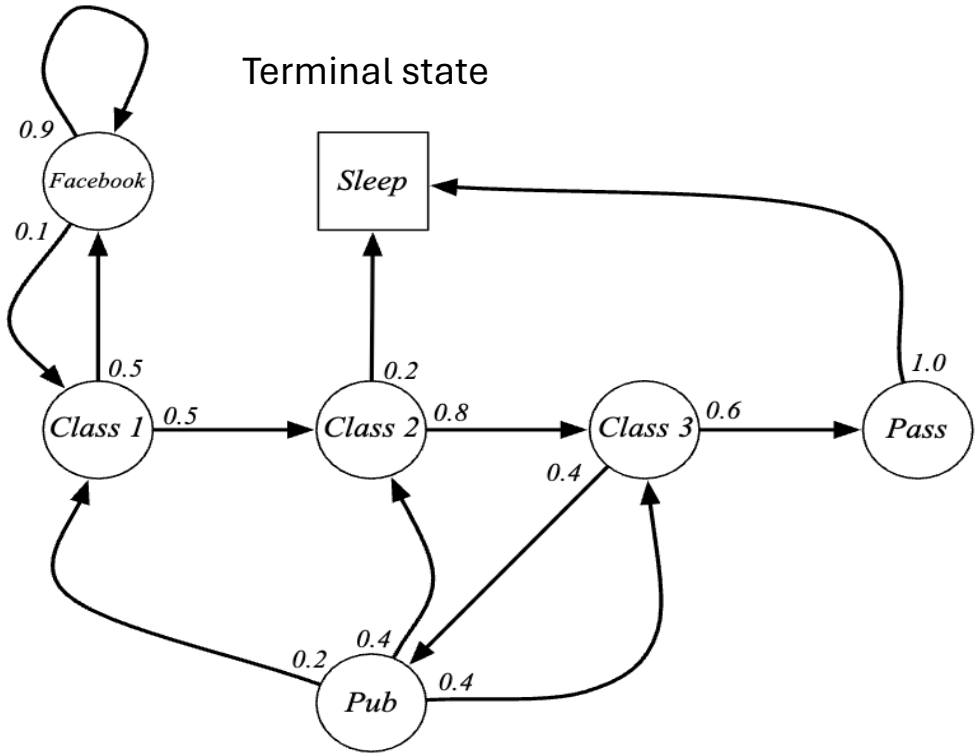
State transition matrix

P

	C1	*C2*	*C3*	*Pass*	*Pub*	*FB*	*Sleep*
C1		0.5				0.5	
C2			0.8				0.2
C3				0.6	0.4		
Pass							1.0
Pub	0.2	0.4	0.4				
FB	0.1					0.9	
Sleep							1

From: D. Silver RL course

Markov process - example



Episodes – finite subsequences of states ending in the terminal state $S_1, S_2, \ldots, S_T$

Sample **episodes**

(S_1 = *Class1*)

- *Class1 Class2 Class3 Pass Sleep*
- *Class1 Facebook Facebook Class1 Class2 Sleep*
- *Class1 Class2 Class3 Pub Class3 Pass Sleep*
- *Class1 Facebook Facebook Class1 Class2 Class3 Facebook Facebook Class1 Class2 Class3 Pub Class2 Sleep*

From: D. Silver RL course

Markov reward processes

A **Markov Reward Process (MRP)** is a Markov chain with numerical values, consisting of a tuple (S, P, R, γ)

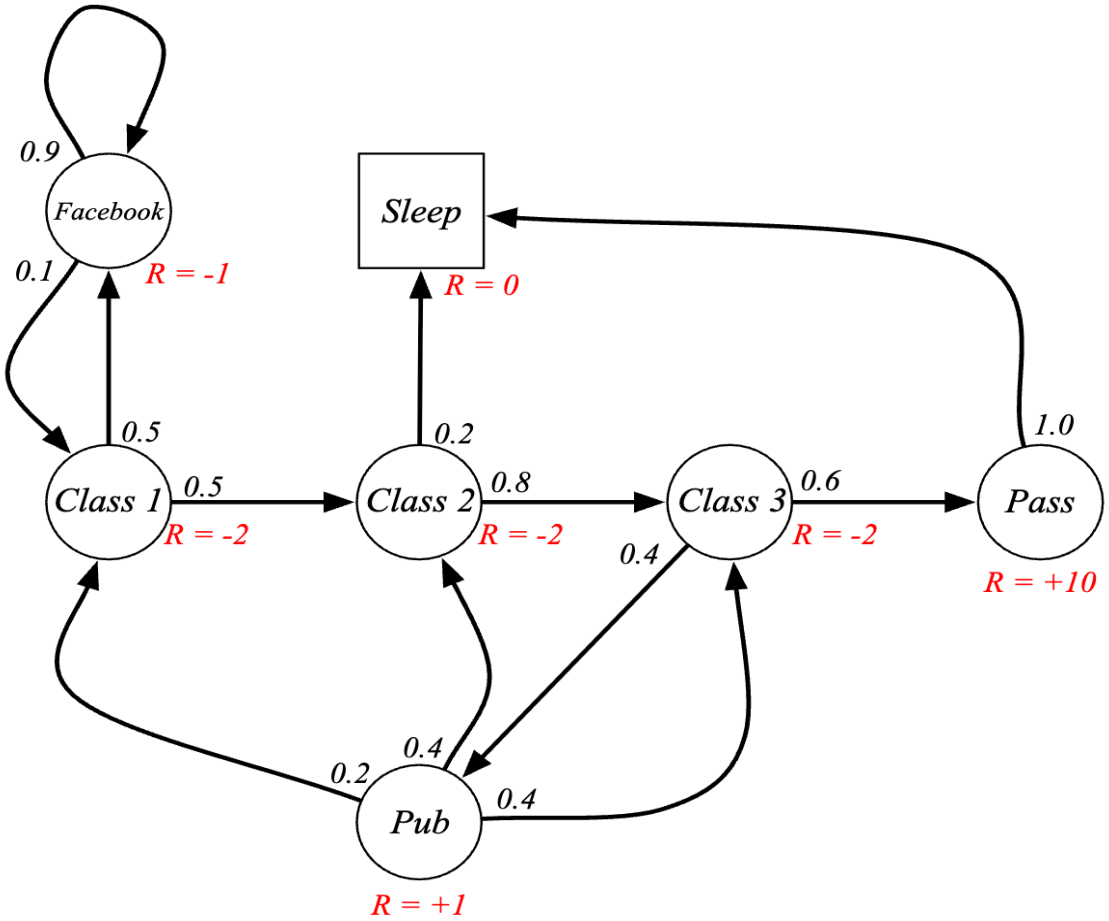
- ***S*** is a finite set of states (as before)
- ***P*** is the state transition probability matrix (as before)
- ***R*** is a reward function associated to the current state

$$R = \mathbb{E}[R_{t+1}|S_t = s]$$

- γ is a discount factor in the range [0,1]

Reward hypothesis – What we mean by goals and purposes can be thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).

Markov reward process - example



From: D. Silver RL course

Return

- The return G_t can be defined as the total discounted reward from time-step t

$$G_t = R_{t+1} + \gamma R_{t+2} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- The discount γ (between 0 and 1) represents how much we value the future rewards in the present – the value of receiving a reward *r* after *k+1* time steps is given by γ^k
- A value of γ equal to one gives all rewards the same value, while values close to zero value current rewards a lot

Why do we use discounts ?

- In most cases MRP (and MDPs) use discounts in the computation of rewards
- In many cases this fits well the **semantics** of the problems (e.g. financial scenarios, reaching goals in the shortest number of steps, human behaviour, uncertainty about future, etc)
- It is Mathematically convenient and avoids infinite returns in cycles
- Better way to deal with uncertainty in the models
- Cognitive model (for humans/ animals)
- We can always use undiscounted variant with $\gamma = 1$ (if we know all episodes end)

Return - example

Calculation of returns for the previous MRP:

$$\gamma = \frac{1}{2}$$

$$G_1 = R_2 + \gamma R_3 + ... + \gamma^{T-2} R_T$$

C1 C2 C3 Pass Sleep	$G_1 =$	$-2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8}$	$=$	-2.25
C1 FB FB C1 C2 Sleep		$-2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16}$	$=$	-3.125
C1 C2 C3 Pub C2 C3 Pass Sleep		$-2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \ldots$	$=$	-3.41
C1 FB FB C1 C2 C3 Pub C1 ...	-	$-2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \ldots$	$=$	-3.20
FB FB FB C1 C2 C3 Pub C2 Sleep				

From: D. Silver RL course

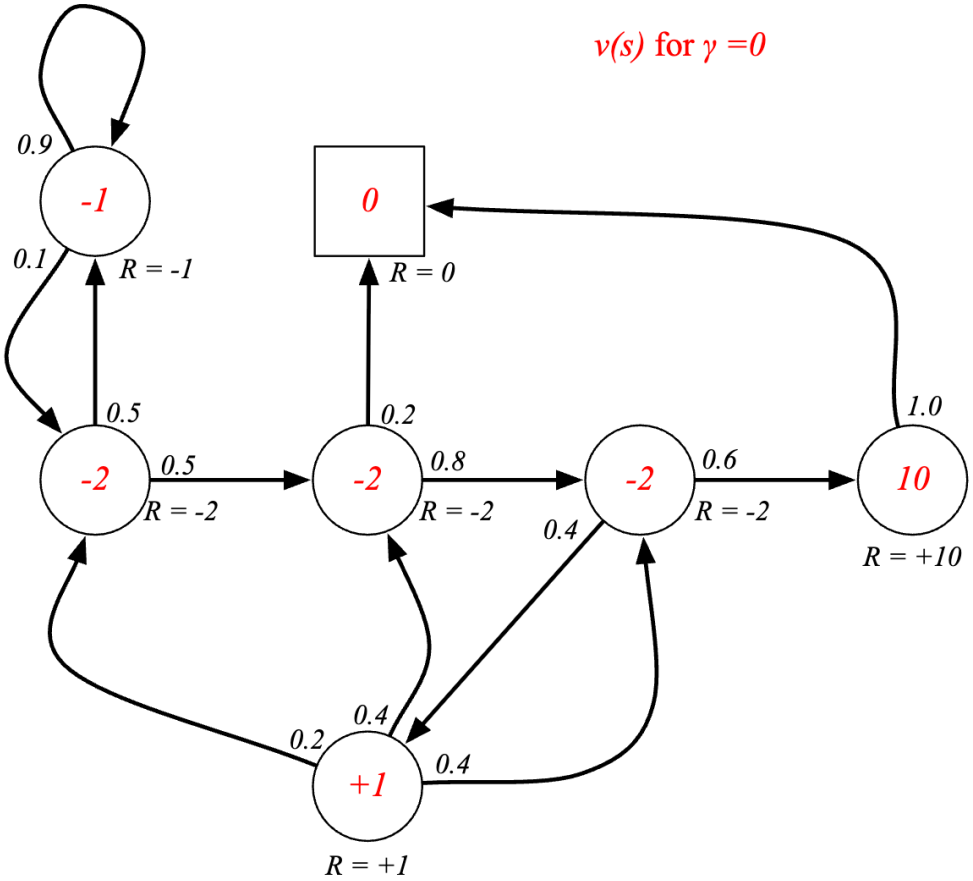
Value function

- The **value function** v(s) provides the long-term value of a state s.
- It may be defined as:

$$v(s) = \mathbb{E}\left[G_t \mid S_t = s\right]$$

- Expected return if you start in this state
- Estimates “how good” it is for the agent to be in that state !!

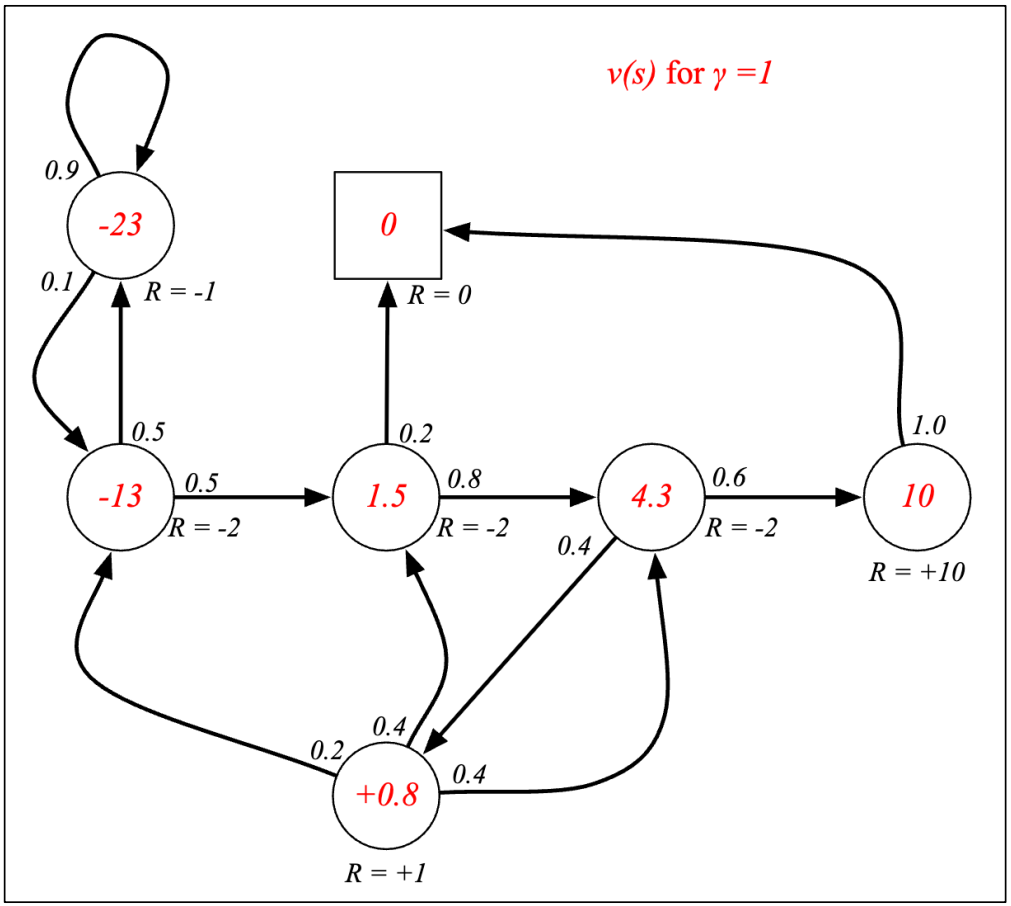
Value function – example

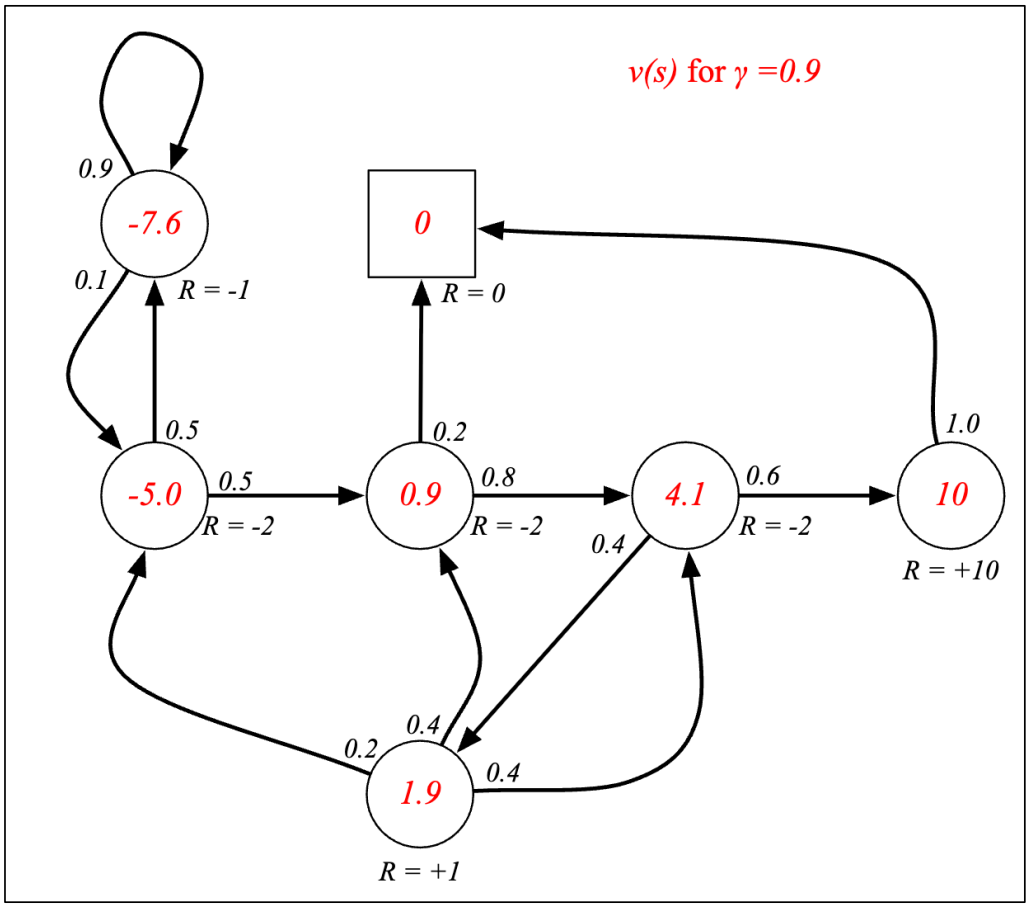


Note that in this case, since this **discount factor is zero**, the value function matches the rewards of each state since only the present state counts for the reward

From: D. Silver RL course

Value function – example

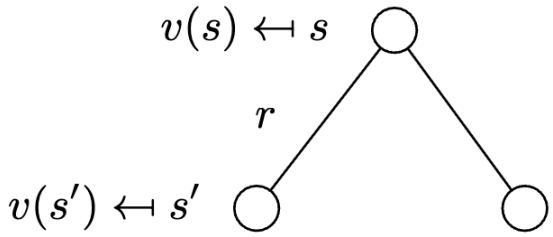




Bellman equation - MRPs

The value function can be decomposed into the
- **immediate reward** and
- the **discounted value of the reward of the following state**

$$
\begin{aligned}
v(s) &= \mathbb{E}\left[G_t \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma\left(R_{t+2} + \gamma R_{t+3} + \ldots\right) \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma G_{t+1} \mid S_t = s\right] \\
&= \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]
\end{aligned}
$$



$$
v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')
$$

Bellman equation in matrix form

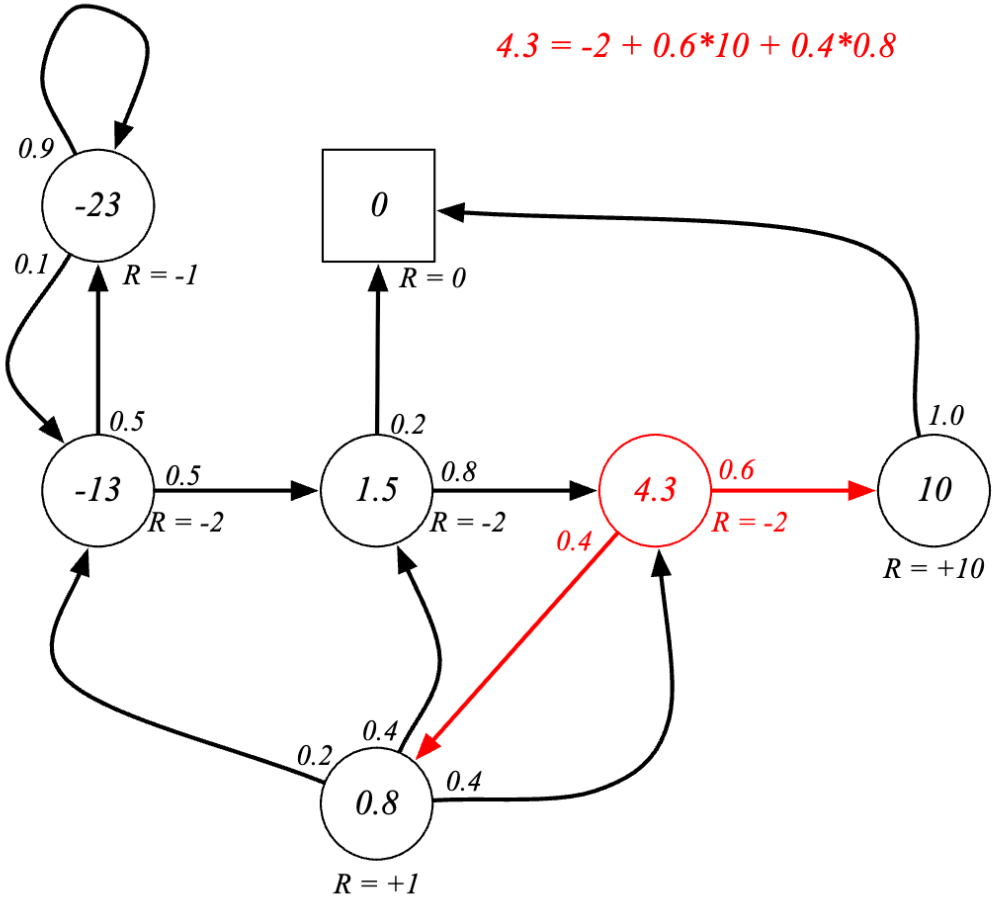
We can express the Bellman equation

$$v = \mathcal{R} + \gamma \mathcal{P} v$$

using matrices and vectors (v is a column vector of length equal to the number of states)

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{11} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Bellman equation – MRPs- example



Verification - calculation of the value function for the previous example (undiscounted)

Bellman equation - MRPs

- The Bellman equation is a linear equation that can be solved directly with complexity $O(n^3)$:

$$\begin{aligned} v &= \mathcal{R} + \gamma \mathcal{P} v \\ (I - \gamma \mathcal{P})\, v &= \mathcal{R} \\ v &= (I - \gamma \mathcal{P})^{-1} \mathcal{R} \end{aligned}$$

- Direct solution only possible for very small problems
- Iterative methods for approximate solutions:
  - Dynamic Programming
  - Monte Carlo
  - Temporal-difference learning

Markov decision processes - MDP

MDPs are represented by tuples **<*S*, *A*, *R*, *P*, γ >**

- ***S*** - finite set of states
- ***A*** – actions that the agent may follow in each state (for now we assume A is the same in all states)
- ***R*** – rewards for each pair (state, action)

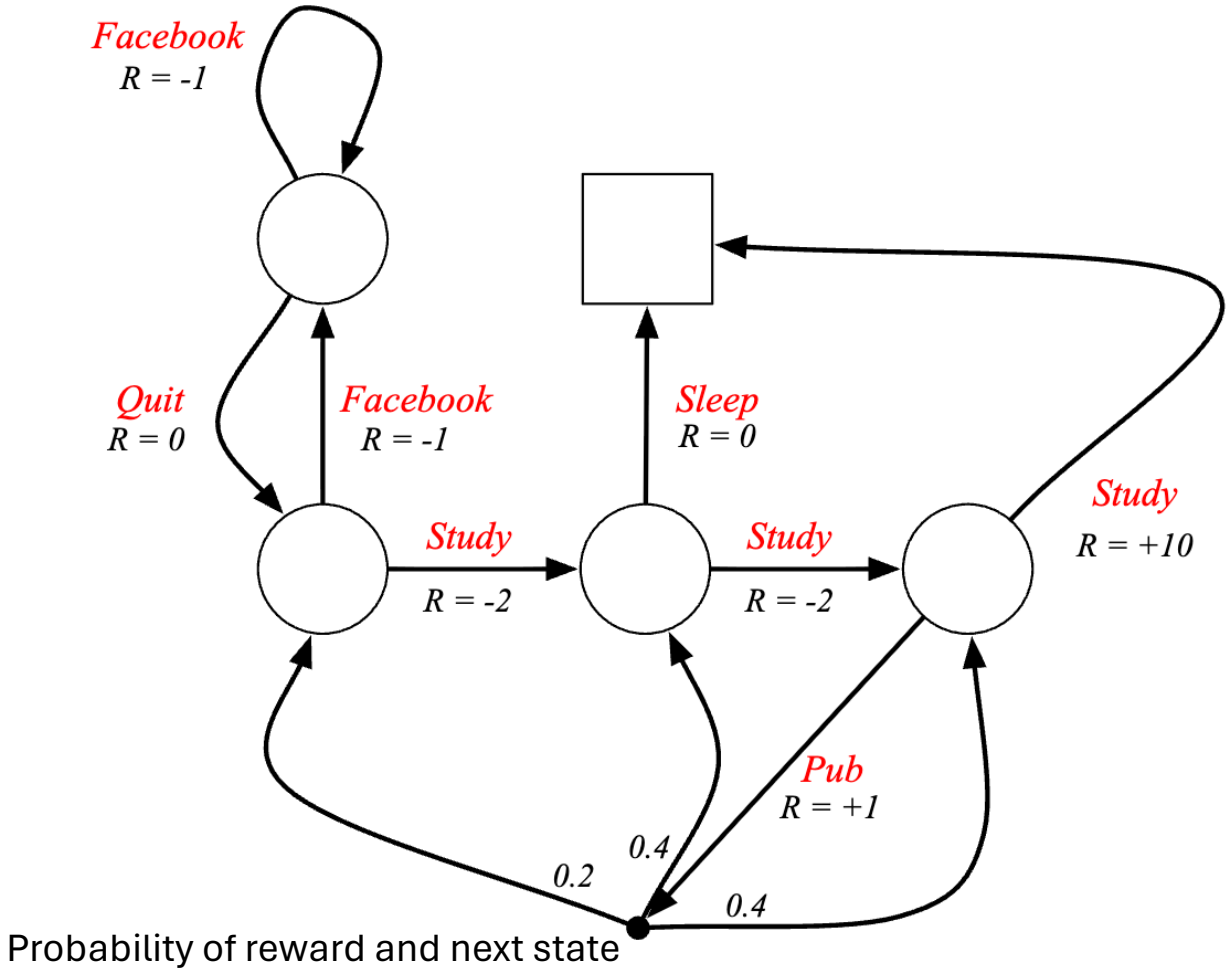
$$\mathcal{R}_s^a = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right]$$

- ***P*** – dynamics of state transitions – probability of next state from pair (previous state, action)

$$\mathcal{P}_{ss'}^a = \mathbb{P}\left[S_{t+1} = s' \mid S_t = s, A_t = a\right]$$

Defines one matrix per action

MDP - example



Probability of reward and next state

Decisions given in red

Assuming equal probabilities for actions leaving each state

Detailed example – recycling robot

- Aim: mobile robot with the job of collecting empty soda cans in an office
- It has sensors for detecting cans; an arm and a gripper to pick them and place them on an on-board bin
- It runs on a rechargeable battery
- Control system has components for interpreting sensory information, for navigating, and for controlling the arm and gripper
- To make the problem simple – the state set has only two **states** related to the battery charge: $S = \{\text{low}, \text{high}\}$

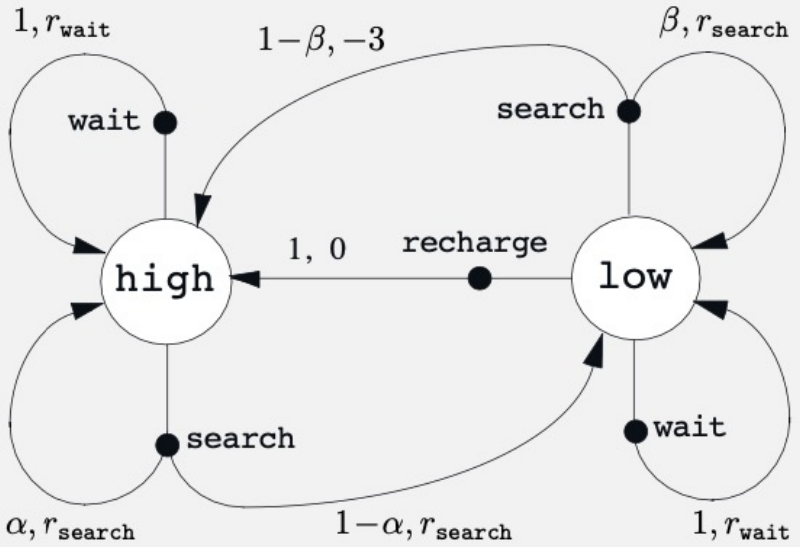
Example from Sutton & Barto

Detailed example – recycling robot

- In each state, the agent can decide whether to (1) actively search for a can, (2) remain stationary and wait for someone to bring it a can, or (3) head back to its home base to recharge its battery.
- When the energy level is high, recharging would always be foolish, so we do not include it in the **action** set for this state. The action sets are: $A(\text{high}) = \{\text{search}, \text{wait}\}$ and $A(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$.
- Rewards are zero most of the time; positive when the robot secures a can, or large and negative if the battery runs all the way down.
- The best way to find cans is to actively search for them, but this runs down the robot's battery, whereas waiting does not.

MDP for the recycling robot

s	a	s'	$p(s' \mid s, a)$	$r(s, a, s')$
high	search	high	α	$r_{\texttt{search}}$
high	search	low	$1-\alpha$	$r_{\texttt{search}}$
low	search	high	$1-\beta$	-3
low	search	low	β	$r_{\texttt{search}}$
high	wait	high	1	$r_{\texttt{wait}}$
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	$r_{\texttt{wait}}$
low	recharge	high	1	0
low	recharge	low	0	-



α - probability of battery going to low when searching if it was high
β - probability of battery being depleted when searching if it was low
Penalty of battery depleted = -3
r_{search} = number of cans collected when searching
r_{wait} = number of cans collected when waiting
0 = reward when recharging

Policies

- A **policy** (denoted by π) defines the **strategy/ behaviour of an agent** – defined by a mapping from states to probabilities of selecting each possible action
- In MDPs, policies **only depend on the current state** and are **time independent**

$$\pi(a|s) = \mathbb{P}\left[A_t = a \mid S_t = s\right]$$

- In an MDP, the state sequence is a Markov sequence/ chain
- The state and reward sequence is a MRP, where

$$\mathcal{P}^{\pi}_{s,s'} = \sum_{a \in \mathcal{A}} \pi(a|s)\mathcal{P}^{a}_{ss'}$$

$$\mathcal{R}^{\pi}_{s} = \sum_{a \in \mathcal{A}} \pi(a|s)\mathcal{R}^{a}_{s}$$

Value function

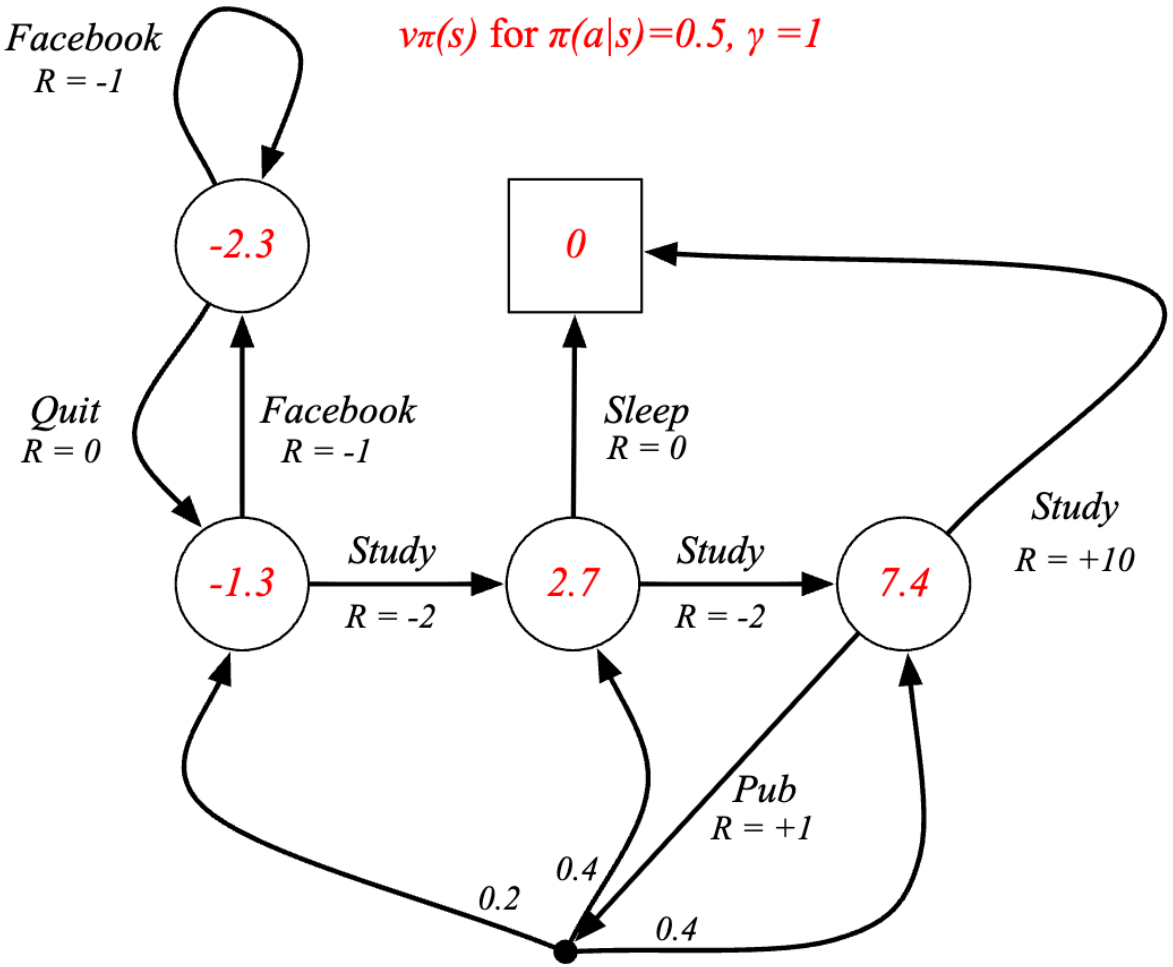
- The **state-value function $v_\pi(s)$** of an MDP and a policy π is the expected return starting from state s and following policy π.

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right], \text{ for all } s \in \mathcal{S}$$

- The **action-value function $q_\pi(s,a)$** is the expected return starting from state s, taking action a, and following policy π

$$q_\pi(s,a) = \mathbb{E}_\pi\left[G_t \mid S_t = s, A_t = a\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a\right]$$

State-value function - example



Bellman expectation equations for MDPs

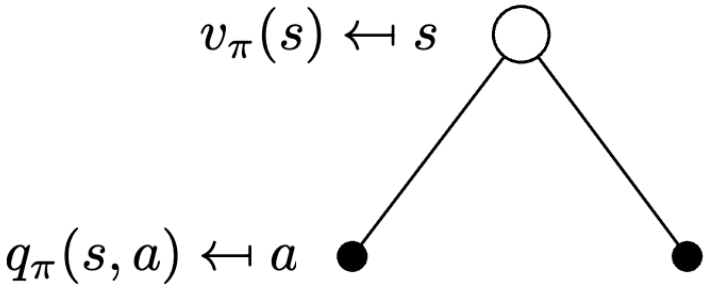
- We can decompose the state-value function into immediate reward + discounted value of next state

$$v_\pi(s) = \mathbb{E}_\pi \left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right]$$

- And the same for the action-value function

$$q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \right]$$

Bellman equations for *V* and *q* functions

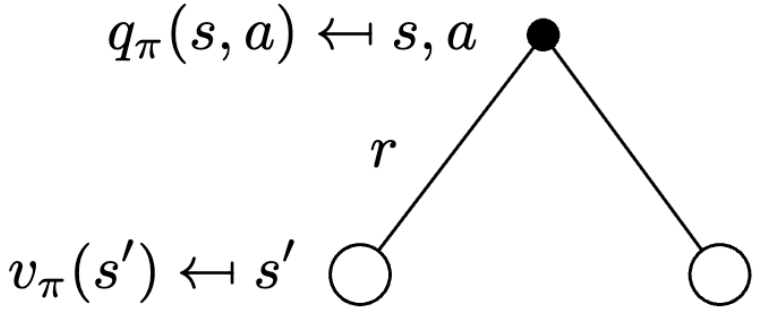


states

actions

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s,a)$$

Value of state - Average over the actions



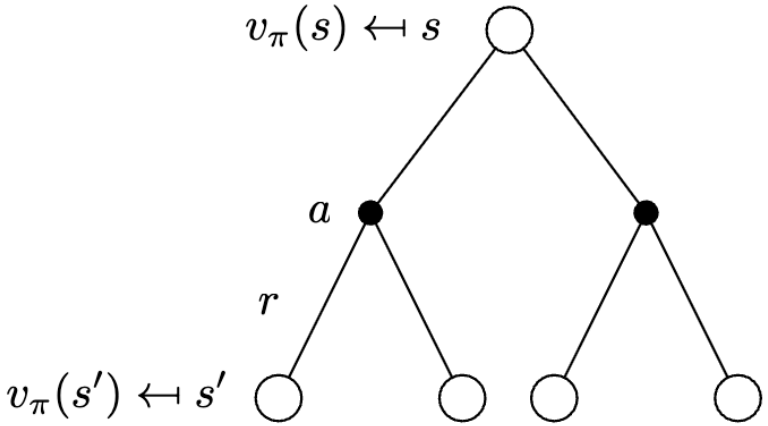
$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

Value of action - Average over the states

Bellman equations for *V* and *q* functions

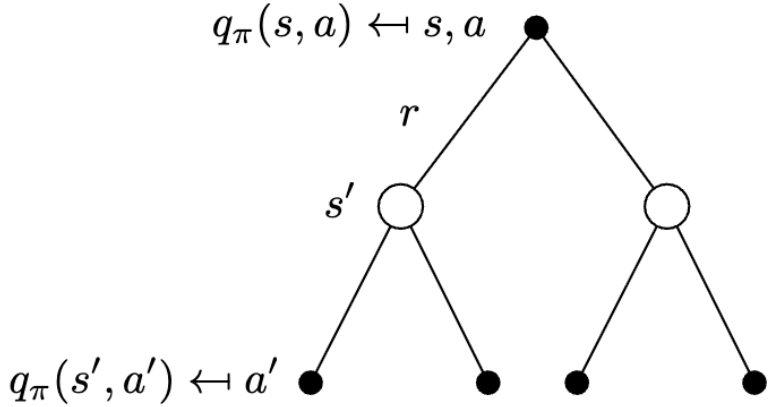
states

actions



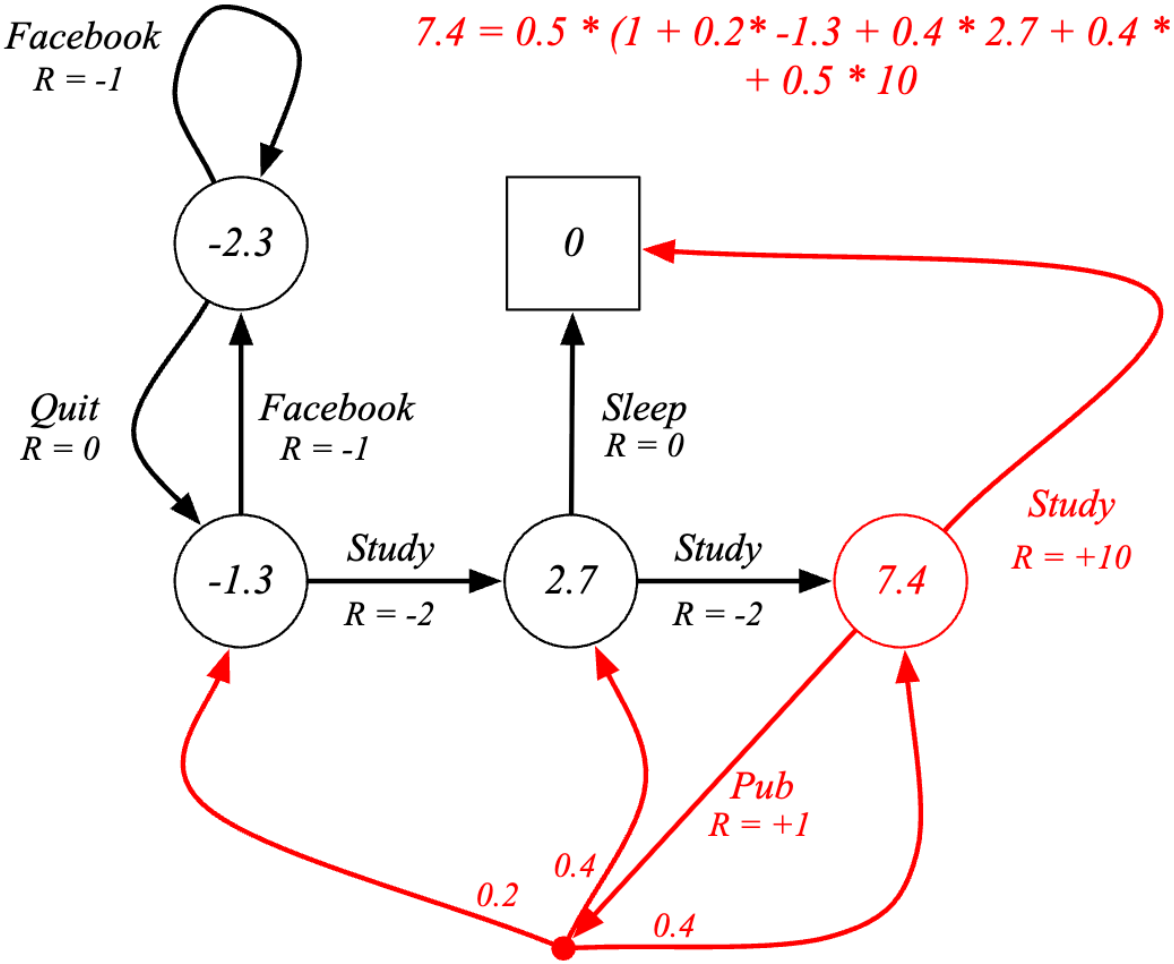
Putting all together

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a')$$

Bellman equation - example



Verifying the calculation

Optimal value function

The **optimal state-value $v_*(s)$** is the maximum value over all policies:

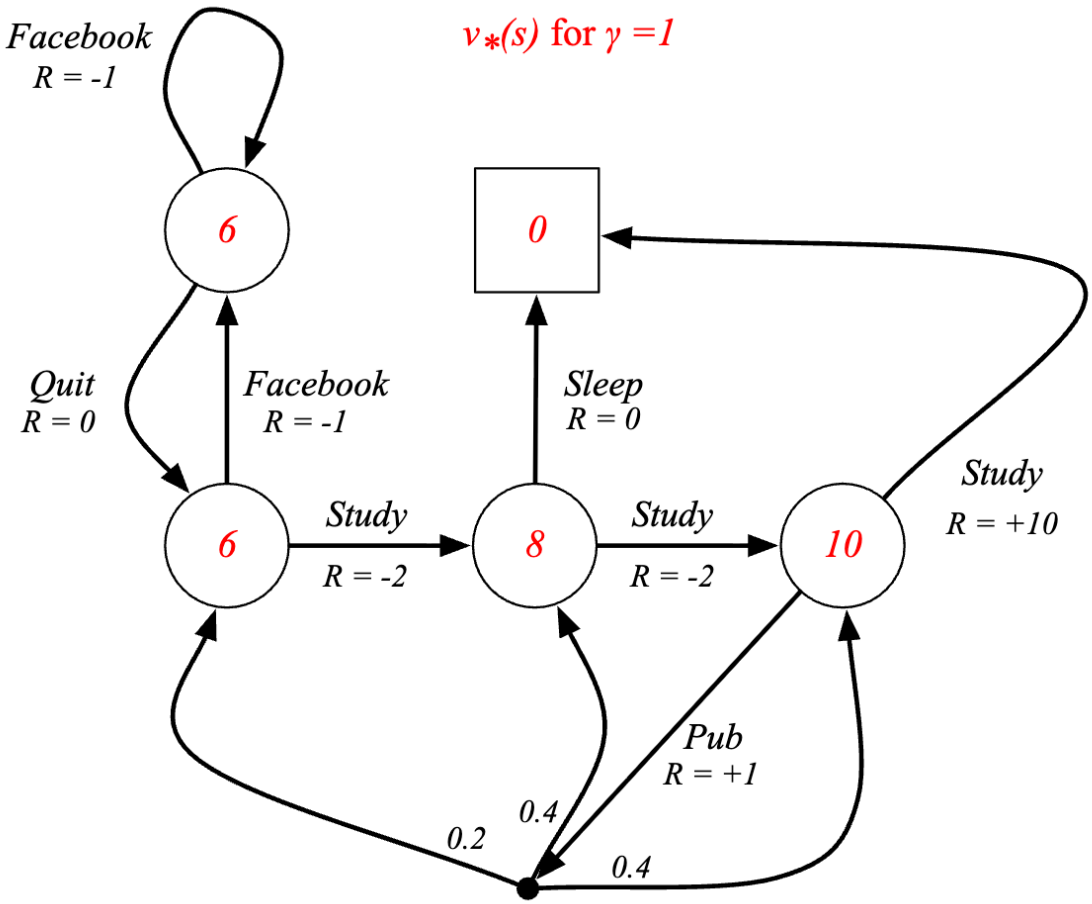
$$v_*(s) = \max_\pi v_\pi(s)$$

The **optimal action-value function $q_*(s,a)$** is the maximum action-value function over all policies

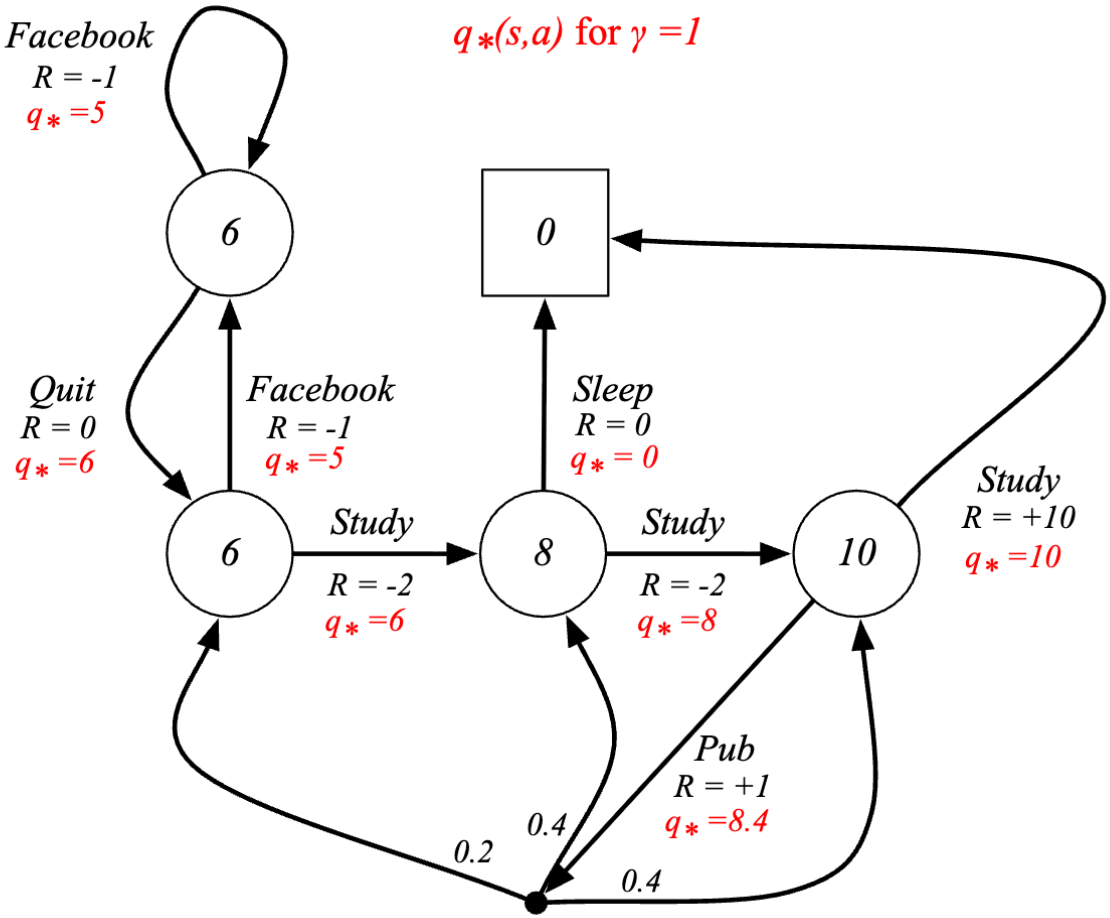
$$q_*(s, a) = \max_\pi q_\pi(s, a)$$

The optimal action-value specifies the policy with the best possible performance – when we know that the MDP is **solved** !!

Optimal value function - example



Optimal action value function - example



Optimal policy

We can define a partial ordering over policies

$$\pi \geq \pi' \text{ if } v_\pi(s) \geq v_{\pi'}(s), \forall s$$

For any MDP, we know that:

- There exists an **optimal policy π_*** that is better than or equal to all other policies (not necessarily unique)
- All optimal policies achieve the optimal value function
- All optimal policies achieve the optimal action-value functions

$$q_{\pi_*}(s, a) = q_*(s, a)$$

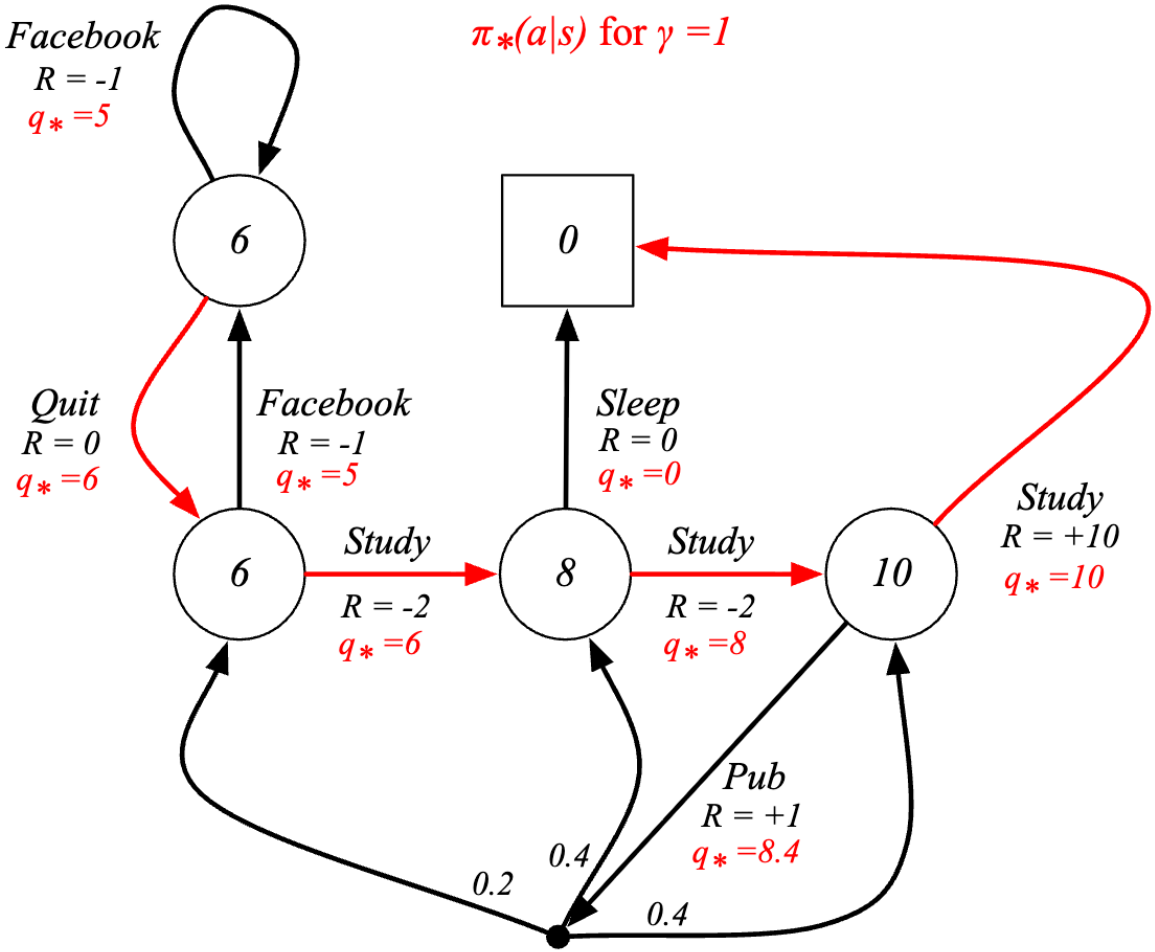
Optimal policy

- An optimal policy can be found by maximising over $q_*(s,a)$

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\text{argmax}}\ q_*(s,a) \\ 0 & \textit{otherwise} \end{cases}$$

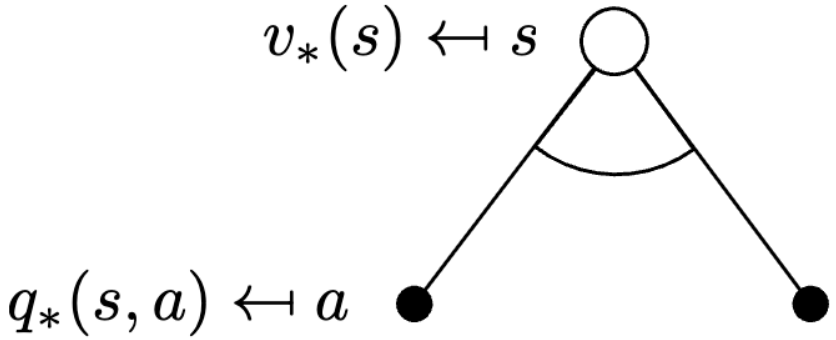
- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s,a)$ we immediately have the optimal policy

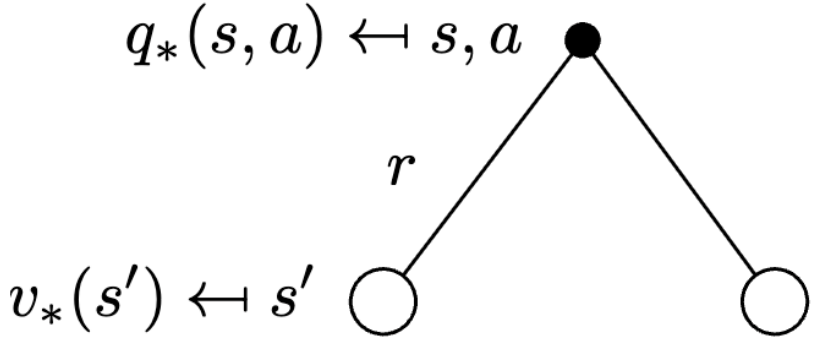
Optimal policy for example



Bellman optimality equation for v and q

The optimal value functions are recursively related by the Bellman optimality equations:



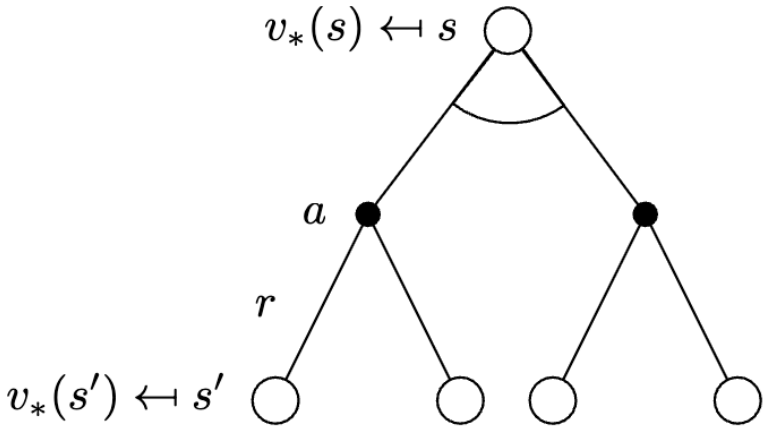


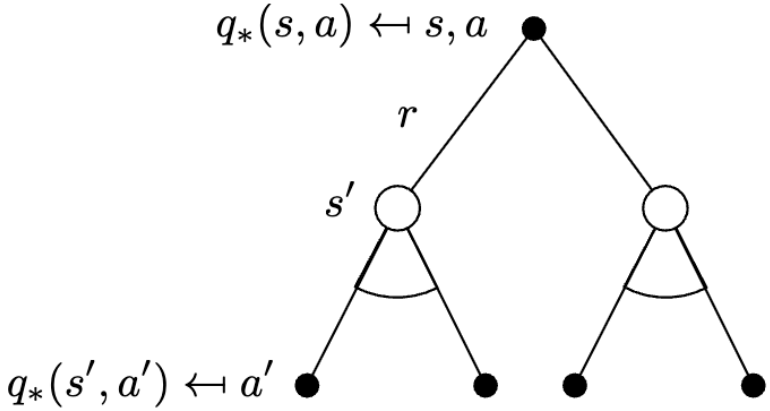
$$v_*(s) = \max_a q_*(s,a)$$

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

Bellman optimality equation for V and q

Putting it all together:

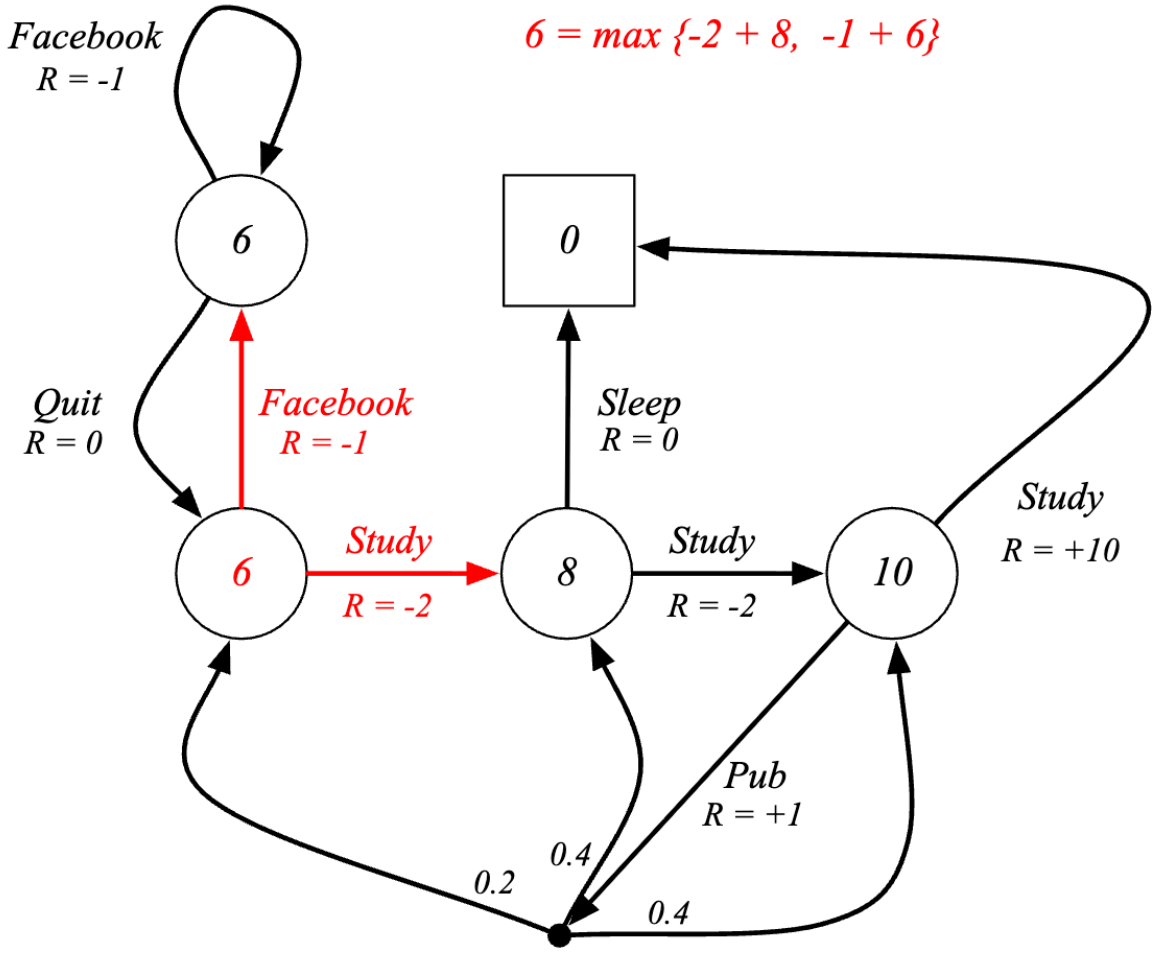




$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s')$$

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a')$$

Bellman optimality equation - example



Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear
- No closed form solution (in general) - it is usually not possible to compute an optimal policy by solving the Bellman optimality equation with reasonable computational costs (cpu, memory)
- Many iterative solution methods (approximation heuristics) will be explored in the course
  - Value Iteration
  - Policy iteration
  - Q-learning
  - Sarsa